

Project Progress Update #2 Speaker Notes (Slide-by-Slide)

Presentation Date: Monday, April 13th, 2026 / Wednesday, April 15th, 2026

Project: Predicting Marathon Race Performance Using Machine Learning

Faculty Mentor: Dr. Hany Elgala

Teaching Assistant: Honglan Chen

Team Members: Sanad Sahawneh, Brian Allen, and Mahib Rahman

Slide 1 — Title (Mahib)

Target time: 15 seconds

“Good morning. We are presenting Project Progress Update #2 for our project, *Predicting Marathon Race Performance Using Machine Learning*. In this update, we moved beyond basic model building and focused on stronger evaluation, better interpretation of results, and a decision-based extension built on top of the prediction system.”

Slide 2 — Agenda (Mahib)

Target time: 15 seconds

“We will begin with a brief recap of the project, then highlight the key achievements since Project Update #1. After that, we will walk through the system pipeline and model implementation, explain how we evaluated the models, present the results and residual analysis, introduce our decision-based over-under system, discuss code structure and team contributions, and then finish with a live demonstration.”

Slide 3 — Brief Project Recap (Brian)

Target time: 35 seconds

“The goal of this project is to predict marathon finish time using machine learning models trained on real race data. We use the 2023 Boston Marathon dataset, which contains about 26,598 runners. Our input features are age group, gender, and half-marathon split time, and the target variable is final finish time in seconds. In Project Update #1, we established the initial modeling framework using linear regression and a neural network. At that stage, the

key insight was that half-marathon split time had a very strong relationship with final finish time, which made this a strong regression problem to study.”

Slide 4 — Key Improvements Since Update #1 (Sanad)

Target time: 50 seconds

“This slide is the most important slide for showing what changed since Project Update #1. First, we introduced a baseline model, which gives us a minimum benchmark and lets us determine whether more advanced models are actually worth using. Second, we expanded our evaluation framework from only MAE and RMSE to also include R-squared and MAPE, so we now assess both absolute and relative model performance. Third, we converted prediction error into minutes, which makes the results much easier to interpret in a real-world setting. We also added a prediction-quality framework, performed residual analysis and error-distribution analysis, improved the neural network architecture to 64, 32, and 16 hidden units, and extended the system into a sports betting-style over-under decision model. So the main point of this update is that the project evolved from simple model comparison into a complete pipeline with benchmarking, interpretation, visualization, and decision support.”

Slide 5 — End-to-End ML Pipeline (Sanad)

Target time: 45 seconds

“This slide shows the full pipeline and begins the code walkthrough. The most important code on this slide is the train-test split. We use `cvpartition` with a holdout of 0.20, which creates an 80 percent training set and a 20 percent testing set. Then we construct `trainData` and `testData` from that split. This matters because it ensures that the models are evaluated on unseen data rather than being judged on the same data they were trained on. Conceptually, the pipeline moves from data loading and cleaning, to splitting, to model training, then to metrics, visualization, and finally decisions. So this code is the foundation that makes the rest of the results meaningful and unbiased.”

Slide 6 — Baseline + Linear Model (Brian)

Target time: 50 seconds

“Here we move into the first two models. The baseline model is important because it is intentionally simple. The key line is `avgRatio = mean(trainData.finish_net_sec ./ trainData.half_time_sec);`. This computes the average ratio between finish time and half-marathon split time in the training set. Then the baseline prediction is produced by multiplying each test runner’s half-marathon split by that average ratio. That gives us a simple estimate of finish time using only the dominant feature.

Next, we train the linear regression model using `fitlm`, with finish time as the target and age group, gender, and half-marathon split time as predictors. The purpose of the linear model is to provide an interpretable machine learning model that still takes multiple features into account. The important conclusion on this slide is that the linear model reduces error relative to the baseline, which confirms that the additional structure captured by the model is useful.”

Slide 7 — Neural Network (Brian)

Target time: 45 seconds

“This slide covers the neural network implementation. The key code is the `fitrnet` call, where we train a neural network using standardized inputs and a hidden-layer structure of 64, 32, and 16. The reason we standardize the inputs is to improve training stability and ensure that different feature scales do not dominate the optimization process. The architecture is designed to let the model capture nonlinear relationships beyond what linear regression can represent. This is important because, while half split is highly predictive, there may still be nonlinear interactions involving age or gender that slightly affect the final prediction. In our results, the neural network performs slightly better than the linear model, but the improvement is modest, which tells us that the problem is already strongly structured and mostly linear.”

Slide 8 — Evaluation Framework (Sanad)

Target time: 45 seconds

“This slide explains how we evaluate model performance. MAE measures average prediction error, so it tells us how far off the model is on average. RMSE also measures error, but it penalizes larger mistakes more heavily, which makes it useful for identifying whether the model occasionally makes big errors. R-squared measures how much variance in finish time is explained by the model, and MAPE expresses error as a percentage, which is useful for relative comparisons across models. The key point here is

that we do not rely on only one metric. Instead, we use a combination of metrics so that model comparison is more complete and more trustworthy.”

Slide 9 — Evaluation Metric Performance Comparison (Mahib)

Target time: 40 seconds

“This slide shows the direct comparison across models. The main result is that the neural network achieves the lowest error across the comparison metrics, but the linear model performs very similarly. The baseline remains useful because it proves that much of the predictive power comes from the strong relationship between half split and final finish time. So, while the neural network is technically the best model, the gap is not dramatic. That tells us two things: first, the problem is highly learnable even with simple models, and second, the more advanced model adds only incremental improvement rather than a dramatic jump.”

Slide 10 — Interpreting Model Performance (Brian)

Target time: 35 seconds

“This slide translates the metrics into practical meaning. Instead of only reporting error in seconds, we convert the error into minutes. That matters because marathon times are naturally understood in minutes and hours, not raw seconds. On this slide, the interpretation framework classifies prediction quality into ranges, and our model falls in the strong range, around seven minutes of MAE. So the real takeaway is that, in practical terms, the system predicts finish time within a reasonable and useful margin for most runners.”

Slide 11 — Results (Brian)

Target time: 45 seconds

“This slide shows actual-versus-predicted plots for both the linear regression model and the neural network. The key code shown here uses scatter to plot actual finish time against predicted finish time, and then overlays a dashed diagonal reference line representing perfect prediction. Points that lie close to this line indicate high accuracy. What we observe is that both models produce point clouds that follow the diagonal very closely, which confirms strong predictive performance. The neural network appears slightly tighter, consistent with the metric results, but visually both models are strong.”

Slide 12 — Visual Analysis: Residuals (Sanad)

Target time: 55 seconds

“This slide goes one level deeper by analyzing residuals. A residual is the actual value minus the predicted value, so it represents the prediction error for an individual runner. The key code first computes `res_lin` and `res_nn` as the difference between `yTest` and each model’s predicted values. Then we plot residual error against predicted finish time and include a horizontal zero line.

What we want in a good residual plot is residuals centered around zero, with no strong visible trend. That is mostly what we see here. The errors are centered around zero, which indicates low bias, meaning the models are not consistently overpredicting or underpredicting. We also notice that the spread becomes a little wider at longer finish times, which suggests that slower runners are somewhat harder to predict. The neural network has a slightly tighter residual spread, which supports the conclusion that it offers a small but real improvement.”

Slide 13 — Decision System: Over/Under Prediction (Sanad)

Target time: 1 minute

“This is the key extension of Project Update #2. Up to this point, the task has been regression: predicting exact finish time. On this slide, we convert that regression output into a binary decision problem. Specifically, we ask whether a runner will finish over or under a chosen threshold, such as 180, 240, or 300 minutes.

The code begins by defining the thresholds in `betLines_min`. Then, for a given threshold, we compare the predicted finish time and the actual finish time to that threshold. That gives us two logical outputs: `predUnder` and `actualUnder`. Finally, we compute decision accuracy as the percentage of cases where those two match.

This is important because it transforms the model from a pure prediction tool into a decision-support tool. The results show about 94 to 97 percent accuracy across thresholds, which means the model is not only good at predicting numeric finish times, but also strong at making practical classification-style decisions based on those predictions.”

Slide 14 — System Demonstration (Sanad)

Target time: 20 seconds before demo

“We will now demonstrate the complete system live. I will run the training script, show the model outputs, display the prediction plots, and then show the over-under decision results so that you can see the full pipeline working in real time.”

Live Demo Script

Target time: 1.5 to 2 minutes

Use this almost word-for-word while demonstrating:

“First, I am opening the main MATLAB script for the project. This script contains the full pipeline we described earlier, including data loading, train-test splitting, baseline prediction, linear regression, neural network training, evaluation, plotting, and the over-under decision analysis.

Now I am running the script. The first output section summarizes the dataset and confirms that the data has been loaded and cleaned correctly.

Next, the script displays the model results table. Here we can directly compare baseline, linear regression, and neural network performance across multiple metrics. This confirms that the neural network has the lowest error, while the linear model remains very competitive.

Now the script generates the actual-versus-predicted plots. As discussed earlier, the points are tightly clustered around the diagonal, which visually confirms strong predictive accuracy.

Next, the residual plots appear. These show that residuals are mostly centered around zero with no strong systematic trend, which supports the conclusion that the models are largely unbiased.

Finally, the script displays the over-under decision results. These results show high decision accuracy across realistic thresholds, which demonstrates that the system can support not only prediction, but also practical decision-making.

So the live demo confirms the full pipeline from raw data to trained model to evaluation to decision output.”

If something loads slowly, say this:

“The plotting stage takes a moment because the script is generating multiple figures and comparisons from the testing set.”

If you need to point at the code while demoing, say this:

“This section handles the data split, this section trains the baseline and linear models, this section trains the neural network, and this final section converts the output into over-under decisions.”

Slide 15 — Team Contributions (Mahib)

Target time: 35 seconds

“Sanad designed and implemented the full machine learning pipeline, developed the evaluation framework, built the decision-based extension, and led the visual analysis and interpretation. Brian focused on data preprocessing, feature selection, and implemented both the linear regression and neural network models. Mahib contributed to feature encoding, normalization, model tuning, and evaluation support. Together, these contributions allowed us to build a complete end-to-end system.”

Slide 16 — Next Steps (Brian)

Target time: 25 seconds

“For future work, we want to improve the model by incorporating additional features such as weather, pacing, and elevation, and by exploring more advanced models and cross-validation. We also want to strengthen the decision system by testing more thresholds and classification-based approaches, and eventually move toward deployment through a simple prediction interface.”

Slide 17 — Questions (Brian)

Target time: 10 seconds

“Thank you for your time. We are happy to take any questions.”